32. Worker Pools, Fan-in/out, Backpressure

32. Worker Pools, Fan-in/out, Backpressure

Phase: Phase 5 — Concurrency
Estimated time: 2 days
Slug: `32-worker-pools`

Learning objectives

- Build a bounded worker pool
- Apply fan-out / fan-in patterns
- Implement backpressure with buffered channels
- Bound concurrency in servers

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Worker pool

```
jobs := make(chan Job)
results := make(chan Result)
for i := 0; i < workers; i++ {
    go worker(ctx, jobs, results)
}
```

Bound the number of goroutines doing expensive work. Unbounded `go` per request is a common outage cause.

Backpressure

When consumers are slow, producers must block or drop according to policy. Buffered channels provide limited slack — not infinite queues.

Fan-out / fan-in

Split work across workers (fan-out), merge results (fan-in) with a WaitGroup + results channel.

Common mistakes (especially from Python instincts)

- Unbuffered infinite job pileup in memory via huge buffers
- No context cancellation on shutdown
- Result channel deadlock when workers exit wrong

Exercises

Work in `exercises/32-worker-pools/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.

- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/32-worker-pools/` until you have an honest failing attempt.

Self-check

1. Why bound workers?
2. What is backpressure?
3. How merge results safely?

Further reading

- https://go.dev/blog/pipelines